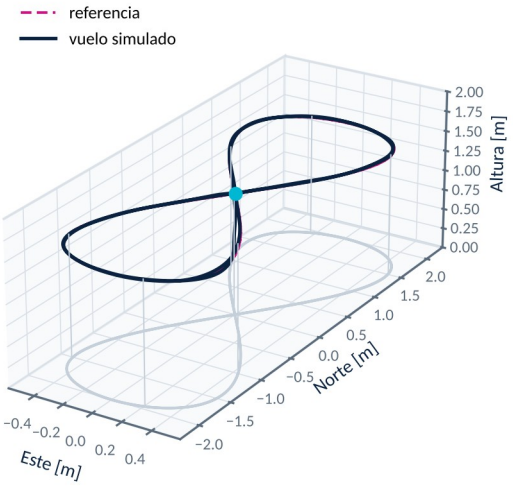
REGISTRO DE SOPORTE LÓGICO · DIRECCIÓN NACIONAL DE DERECHO DE AUTOR

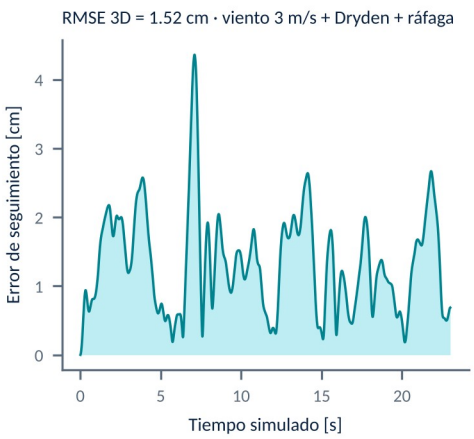
QuadSIM-PRIMBIO

Simulador 6-DOF de cuadricóptero para sintonizar controladores en un entorno simulado, como punto de partida de la sintonía de drones reales

DESCRIPCIÓN DEL SOFTWARE

Información general del producto, requisitos, funcionalidades, calidad, validación y contribución al estado del arte.





Corrida real del simulador: MPC lineal sobre la figura en 8 con viento, PAE-X450, semilla 1.

DOCUMENTO	CÓDIGO	REVISIÓN
Descripción del Software	**PQS-DS-001**	**3.0**
SOFTWARE	FECHA	ESTADO
QuadSIM-PRIMBIO v3.0	**Octubre de 2026**	**Emitido para registro**
AUTOR(ES)	MODALIDAD	IDIOMA
Jesús Daniel Jácome (autor) · Francisco Javier Burgos Flórez (director)	**Proyecto Académico Especial (PAE)**	**Español**
AFILIACIÓN	REPOSITORIO	
Universidad Nacional de Colombia, Sede La Paz · Semillero PRIMBIO · Grupo de Investigación IAM-NANO	**github.com/JesusJacome18/PAE-Quadcopter-SIM**	

Documento técnico de soporte para el registro de soporte lógico ante la Dirección Nacional de Derecho de Autor (Colombia). Las cifras de este documento provienen de ejecuciones del propio software y pueden reproducirse con los guiones del repositorio.

CONTROL DEL DOCUMENTO

Título	QuadSIM-PRIMBIO — Descripción del Software
Código / revisión	PQS-DS-001 · Rev. 3.0
Versión del software	3.0 (repositorio github.com/JesusJacome18/PAE-Quadcopter-SIM)
Elaboró	Jesús Daniel Jácome (autor) · Francisco Javier Burgos Flórez (director)
Revisó	Francisco Javier Burgos Flórez (director)
Contexto	Proyecto Académico Especial (PAE) de Ingeniería Mecatrónica, desarrollado en el Semillero de Investigación PRIMBIO, adscrito al Grupo de Investigación IAM-NANO
Clasificación	Uso académico. Documento de soporte para el registro de soporte lógico ante la DNDA.

HISTORIAL DE REVISIONES

Rev.	Fecha	Descripción del cambio	Elaboró
3.0	2026-10	Emisión inicial para el registro ante la DNDA. La numeración de la revisión coincide con la versión 3.0 del software.	Jesús Daniel Jácome

DOCUMENTOS DEL REGISTRO

Código	Documento	
PQS-DS-001	Descripción del Software	este documento
PQS-MT-002	Manual Técnico	
PQS-MU-003	Manual de Usuario	
PQS-TF-004	Título y Descripción de Funciones	

Contenido

1 Información general del producto....4
2 Descripción general del software....4
2.1 Especificación de requisitos de hardware y software....5
2.2 Justificación de los requisitos....6
3 Objetivos y área de aplicación....6
3.1 Objetivo general....6
3.2 Objetivos específicos....6
3.3 Área de aplicación....7
4 Funcionalidades principales....7
5 Adaptabilidad, extensibilidad y mantenibilidad....8
5.1 Modularidad....8
5.2 Registros como contrato....8
5.3 Vehículos nuevos....8
5.4 Barridos extensibles....8
5.5 Mantenimiento....8
6 Robustez, desempeño y consistencia....8
6.1 Robustez....8
6.2 Desempeño....9
6.3 Consistencia y reproducibilidad....9
7 Interfaz y usabilidad....9
8 Integridad y seguridad de la información....10
9 Portabilidad y compatibilidad....11
9.1 Plataformas compatibles....11
9.2 Dependencias relevantes....11
9.3 Portabilidad....11
10 Documentación y soporte técnico....11
10.1 Soporte técnico....12
11 Pruebas, validación y desempeño....12
11.1 Pruebas automáticas....12
11.2 Verificación numérica del modelo....13
11.3 Banco de comparación de controladores....13
12 Impacto y utilización....14
12.1 Docencia....15
12.2 Investigación....15
12.3 Extensión....15
12.4 Usuarios y utilización....15
13 Contribución al estado del arte....15
13.1 Problema abordado....15
13.2 Estado del arte....15
13.3 Limitaciones actuales del estado del arte....16
13.4 Aporte de {software}....
14 Aportes de los autores....16
15 Referencias....17

1 Información general del producto

Tabla 1 | Ficha del producto.

Nombre del software	QuadSIM-PRIMBIO
Nombre del repositorio	PAE Quadcopter SIM (nombre de desarrollo del proyecto, que se conserva en el código fuente)
Versión	3.0
Tipo de producto	Software científico y académico de simulación: simulador de escritorio de un cuadricóptero con modelo físico no lineal, sensores, estimación, control en cascada, controladores de trayectoria y herramientas de sintonía automática.
Área de aplicación	Control automático, dinámica de vehículos aéreos no tripulados, robótica móvil, instrumentación y filtrado digital, estimación de estado, control predictivo, aprendizaje por refuerzo aplicado al control y simulación numérica.
Autor(es)	Jesús Daniel Jácome (autor) · Francisco Javier Burgos Flórez (director)
Contacto	Jesús Daniel Jácome: jejacome@unal.edu.co · Francisco Javier Burgos Flórez: fjburgosf@unal.edu.co
Afiliación institucional	Universidad Nacional de Colombia, Sede La Paz (Valledupar, Cesar). Semillero de Investigación PRIMBIO (Plataformas Robóticas Inteligentes Interconectadas), Grupo de Investigación IAM-NANO.
Modalidad	Proyecto Académico Especial (PAE) de Ingeniería Mecatrónica, desarrollado en el Semillero de Investigación PRIMBIO, adscrito al Grupo de Investigación IAM-NANO.
Año de desarrollo	2026
Estado del software	Funcional y verificado. Versión 3.0 con modelo 6-DOF no lineal, 5 controladores de posición, perturbaciones, barridos automáticos y aprendizaje por refuerzo fuera de línea; 223 pruebas automáticas y un guion de verificación numérica del modelo.
Lenguaje de programación	Python 3.10 o superior. Núcleo de simulación sobre NumPy; interfaz gráfica con PySide6 (Qt 6), pyqtgraph y PyOpenGL.
Plataformas	Aplicación de escritorio multiplataforma (Windows, Linux). Las pruebas automáticas y las capturas de este documento se ejecutaron en Linux.
Hardware asociado	Ninguno. El software es solo simulación; el vehículo de referencia (PAE-X450, clase F450 de 1.4 kg) se describe con parámetros publicados en la literatura.
Repositorio de código fuente	github.com/JesusJacome18/PAE-Quadcopter-SIM
Documentación asociada	Manual Técnico (PQS-MT-002), Manual de Usuario (PQS-MU-003), Título y Descripción de Funciones (PQS-TF-004) y documento del modelo matemático, 3.ª edición (55 páginas).

2 Descripción general del software

QuadSIM-PRIMBIO es un simulador de un cuadricóptero pensado para una tarea concreta: sintonizar controladores en un entorno simulado y llevar esa sintonía, como punto de partida, a un dron real. El programa reproduce el vehículo como un sólido rígido de seis grados de libertad con cuatro rotores de dinámica propia, mide su estado con sensores que tienen ruido, sesgo, retardo y vibración, estima la actitud con tres algoritmos intercambiables y cierra los lazos de control con la misma arquitectura en cascada que usa el autopiloto PX4.

Ajustar un dron en hardware es lento y caro. Cada cambio de ganancia exige compilar, cargar el firmware, armar el vehículo y volar, y un error puede terminar en hélices rotas. Las herramientas de simulación más difundidas resuelven otro problema: ofrecen escenarios fotorrealistas o integración con ROS, pero rara vez muestran, dentro de la misma ventana, el espectro del giroscopio, el Bode de los filtros, el error del estimador y las métricas de un escalón. Este software junta esas piezas y agrega lo que se necesita para comparar con rigor: semillas fijas, escenarios estándar de perturbación y barridos automáticos de parámetros.

La arquitectura separa un núcleo de simulación escrito solo con NumPy (paquete `quadsim/`, sin dependencias de interfaz) de una interfaz gráfica de escritorio construida con PySide6 (paquete `gui/`). El núcleo se usa igual desde la interfaz, desde los guiones de experimentos y desde las pruebas automáticas, de modo que lo que se ve en pantalla y lo que se publica en una tabla salen del mismo código. La física avanza a 1 kHz con Runge–Kutta de cuarto orden; el lazo de control corre a 500 Hz y el lazo de posición a 100 Hz. La interfaz solo observa: su temporizador de 50 Hz y el redibujo de las gráficas no cambian la física (figura 1).

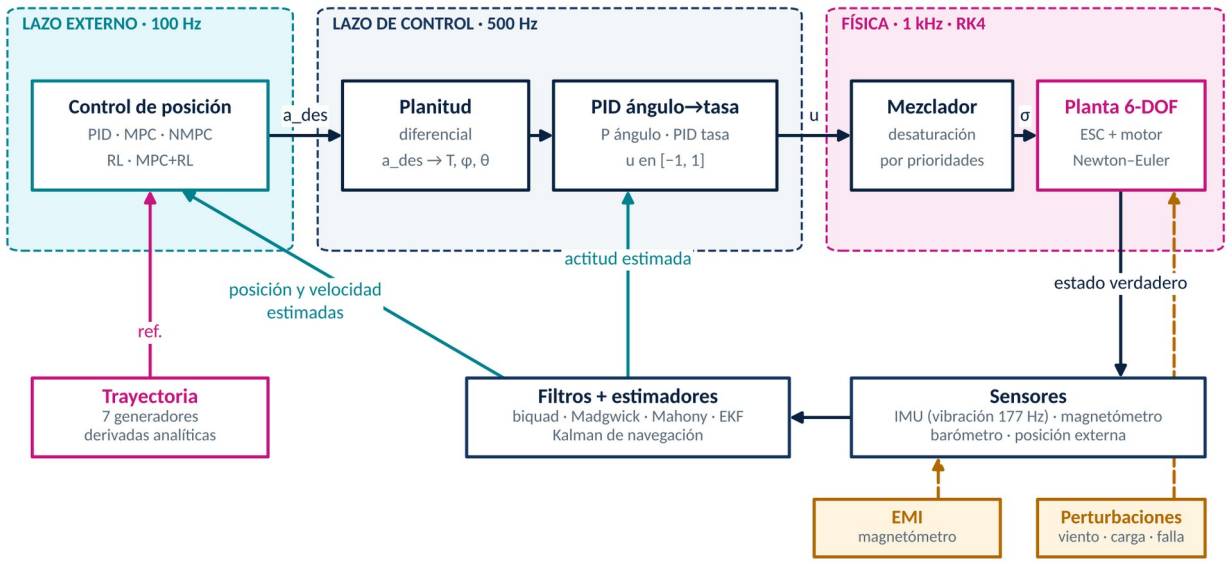


Figura 1 | Estructura de lazos del simulador y tasa a la que corre cada bloque. En magenta la referencia; en cian lo estimado; en ámbar las perturbaciones.

Los usuarios previstos son estudiantes de Ingeniería Mecatrónica y de áreas afines que aprenden control de multirrotores, integrantes del semillero que comparan técnicas de control y estimación, y quien prepara la sintonía inicial del dron del proyecto antes de volarlo.

2.1 Especificación de requisitos de hardware y software

Los valores de la tabla 2 se midieron en la máquina de desarrollo. No son un mínimo absoluto: indican con qué recursos se verificó el funcionamiento.

Tabla 2 | Requisitos de hardware y valores medidos.

Recurso	Requisito	Referencia medida
Procesador	x86-64, 2 núcleos o más	Intel Xeon 2.8 GHz, 2 núcleos: simulación sin interfaz a ≈13× tiempo real con el estimador Mahony
Memoria RAM	1 GB libre	Interfaz en uso: ≈200 MB residentes; núcleo sin interfaz: ≈32 MB
Disco	≈1.5 GB	Repositorio sin historial git: 36 MB; dependencias de Python: ≈0.9 GB (PySide6 ocupa 650 MB)
Pantalla	1450 × 900 px lógicos o más	Capturas tomadas a 1600 × 1000
Tarjeta gráfica	Opcional (OpenGL)	Sin OpenGL la vista 3D pasa a una proyección isométrica 2D
Red	No requerida	Solo para instalar dependencias

Tabla 3 | Requisitos de software.

Componente	Versión mínima	Versión verificada	Uso
Python	3.10	3.13	Intérprete
NumPy	1.24	2.5	Núcleo numérico (única dependencia de `quadsim/`)
PySide6	6.5	6.11	Interfaz gráfica (Qt 6)
pyqtgraph	0.13.3	0.14	Gráficas en tiempo real
PyOpenGL	3.1	3.1.10	Vista 3D (opcional)
SciPy	1.10	1.18	Identificación del modelo sustituto de RL
Matplotlib	3.7	3.11	Gráficas de barridos fuera de la interfaz
pytest	7	9.1	Pruebas automáticas

2.2 Justificación de los requisitos

La decisión que más pesa sobre los requisitos es que el núcleo dependa solo de NumPy. Un simulador que se va a usar para sintonizar tiene que poder correr miles de veces sin interfaz, en procesos paralelos y en máquinas modestas; atarlo a un motor de videojuego o a ROS habría encarecido cada una de esas corridas. Con NumPy el modelo completo avanza alrededor de doce veces más rápido que el tiempo real en un solo núcleo, suficiente para que un barrido de cien simulaciones termine en uno o dos minutos.

PySide6 y pyqtgraph se eligieron por rendimiento: las gráficas reciben arreglos de NumPy sin copiarlos a listas y se redibujan a 30 cuadros por segundo con miles de puntos. OpenGL se dejó opcional porque los equipos de laboratorio y las máquinas virtuales no siempre lo tienen; sin él la interfaz sigue funcionando. SciPy y Matplotlib solo intervienen en tareas fuera de la interfaz (identificación del modelo sustituto y gráficas de los barridos).

3 Objetivos y área de aplicación

3.1 Objetivo general

Desarrollar un simulador de cuadricóptero con un modelo físico verificable que permita sintonizar, comparar y validar técnicas de control y estimación en un entorno simulado, como punto de partida para la sintonía de drones reales.

3.2 Objetivos específicos

1. Modelar la dinámica no lineal de seis grados de libertad del vehículo, su propulsor, sus sensores y las perturbaciones principales (viento, interferencia magnética, carga útil, falla de motor), con parámetros trazables a su fuente.
2. Reproducir el control de bajo nivel de un autopiloto (PID en cascada ángulo–tasa) con ganancias iniciales calculadas desde el modelo.
3. Comparar en igualdad de condiciones cinco controladores de posición: PID, MPC lineal, MPC no lineal, aprendizaje por refuerzo puro e híbrido MPC con aprendizaje por refuerzo.
4. Automatizar la sintonía de filtros, estimadores y ganancias mediante barridos de parámetros con criterios cuantitativos.
5. Ofrecer una interfaz gráfica didáctica en español que muestre, en vivo, el efecto de cada parámetro.
6. Respaldar cada resultado con pruebas automáticas y verificación numérica reproducible.

3.3 Área de aplicación

El software se ubica entre el control automático y la robótica aérea. Sirve como laboratorio virtual en cursos de control, instrumentación y sistemas dinámicos; como banco de pruebas para trabajos de investigación del semillero sobre control predictivo, estimación de estado y aprendizaje por refuerzo; y como herramienta de ingeniería para obtener ganancias y filtros iniciales de un multirrotor antes de probarlo en vuelo.

4 Funcionalidades principales

La tabla 4 resume las funciones del programa. El Manual de Usuario describe cómo se usa cada una desde la interfaz y el Manual Técnico cómo están programadas.

Tabla 4 | Funcionalidades del software.

Función	Descripción
Simulación física 6-DOF	Modelo Newton–Euler no lineal de 17 estados (posición, velocidad, cuaternión, velocidad angular y velocidad de los 4 rotores) en marcos NED/FRD, integrado con RK4 a 1 kHz. Incluye efecto giroscópico de los rotores, arrastre del fuselaje, amortiguamiento rotacional y efecto suelo opcional.
Catálogo de vehículos	5 vehículos con su fuente bibliográfica. Por defecto el PAE-X450 (F450 en X, 1.4 kg) con parámetros de CopterSim/RflySim.
Propulsor y mezclador	ESC + motor sin escobillas de primer orden, empuje y par cuadráticos y asignación de control con desaturación por prioridades.
Sensores	IMU con ruido de hoja de datos, sesgo, deriva y vibración a la frecuencia de paso de pala; magnetómetro, barómetro y sensor de posición externo, cada uno con su tasa y su retardo.
Filtros digitales	Dos biquads en serie por eje (pasa-bajos, Butterworth, Chebyshev, notch) con su diagrama de Bode y el espectro del giroscopio en vivo.
Estimación	Madgwick, Mahony y filtro de Kalman extendido de 7 estados, intercambiables en vuelo; filtro de Kalman de navegación para posición y velocidad.
Control de actitud	PID en cascada ángulo → tasa → momento normalizado, con ganancias SIMC calculadas del modelo y prueba de escalón con métricas automáticas.
Trayectorias	7 generadores con derivadas analíticas: vuelo estacionario, escalones (caja), círculo, figura en 8 (lemniscata), hélice, Mínimo snap (waypoints), punto a punto (mínimo jerk).
Controladores de posición	PID en cascada (estilo PX4), MPC lineal con previsión, MPC no lineal (iLQR), aprendizaje por refuerzo puro (PPO) e híbrido MPC + RL residual.
Comparación de controladores	Corrida de varios controladores sobre la misma trayectoria, con la misma semilla de ruido y turbulencia; tabla de RMSE, error máximo, esfuerzo, saturación y costo de cómputo.
Perturbaciones	Viento medio, turbulencia de Dryden, ráfaga 1−cos, interferencia electromagnética en el magnetómetro, carga útil con cambio de masa, inercia y centro de masa, y pérdida de eficiencia de un motor; inmediatas o programadas.
Barridos de parámetros	8 barridos predefinidos de filtros, estimadores y PID; diseños en rejilla, aleatorio, hipercubo latino y refinamiento local; costo normalizado, frente de Pareto, sensibilidad, ejecución paralela y reanudable.
Aprendizaje por refuerzo fuera de línea	Identificación de un modelo sustituto rápido, entorno vectorizado con aleatorización de dominio, PPO programado en NumPy y validación en el simulador completo.
Visualización y exportación	Gráficas en vivo, horizonte artificial, brújula, barras de motores, dial de viento, vista 3D; exportación de la corrida a CSV a 100 Hz.
Documentación del modelo	Página con cada parámetro del vehículo, su unidad y su fuente, cantidades derivadas, relojes de la simulación y ecuaciones.

La interfaz incluye además ayuda integrada: un tour guiado de 37 pasos (F1), una galería de 11 ejemplos listos para cargar, cada uno con su resultado esperado y la opción de guardar y abrir escenarios completos.

5 Adaptabilidad, extensibilidad y mantenibilidad

5.1 Modularidad

Cada fenómeno físico o algoritmo vive en su propio módulo de `quadsim/`: dinámica, propulsión, sensores, filtros, estimadores, perturbaciones, control, trayectorias, barridos y aprendizaje por refuerzo. El orquestador `simulator.Simulation` los une con tres relojes y expone una interfaz pública corta (`reset`, `step_control`, `run`, `telemetry`, `set_estimator`, `set_filter`, `set_position_controller`). La interfaz gráfica nunca toca el simulador directamente: lo hace a través de un único objeto coordinador, `SimulationController`.

5.2 Registros como contrato

Las trayectorias y los controladores de posición se registran en diccionarios (`TRAJECTORIES`, `CONTROLLERS`). Cada clase declara un esquema `PARAMS` con clave, etiqueta, valor por defecto, límites, paso y unidad. La interfaz construye sus formularios a partir de ese esquema, así que un parámetro nuevo aparece en pantalla sin modificar la GUI. Agregar un controlador consiste en heredar de `PositionController`, escribir su método `update()` y añadir la clase al registro.

5.3 Vehículos nuevos

Un vehículo es un objeto `VehicleParams` inmutable con masa, brazo, inercias, coeficientes de empuje y par, recta del ESC y constante de tiempo del motor, más su fuente y sus supuestos. Cuando se midan los parámetros del dron real del proyecto bastará con declarar un vehículo nuevo en el catálogo; las ganancias SIMC, los momentos máximos y el acelerador de vuelo estacionario se recalculan solos.

5.4 Barridos extensibles

Los barridos nombran cada parámetro con una ruta de texto (por ejemplo `filter.rp.0.fc` o `estimator.mahony.kp`). Con esa gramática se puede barrer cualquier campo de la configuración, del vehículo, de los sensores o de las perturbaciones sin escribir código nuevo.

5.5 Mantenimiento

El código está comentado en español y explica el porqué de cada decisión, con la fuente (autor, año, ecuación o página) de cada número tomado de la literatura. La batería de 223 pruebas automáticas cubre trayectorias, controladores, barridos, aprendizaje por refuerzo y la interfaz sin pantalla; el guion `experiments/model_verification.py` recalcula las cifras del documento del modelo y las guarda en un archivo JSON.

6 Robustez, desempeño y consistencia

6.1 Robustez

- **La interfaz no se cae por un error del núcleo.** Toda excepción durante la simulación pausa la corrida y se informa en el registro de eventos con su traza; lo mismo ocurre al cambiar vehículo, estimador o controlador.
- **Controladores que no cargan.** Si un controlador de aprendizaje no encuentra sus pesos, el registro muestra el error, la simulación vuelve al modo actitud y el resto del programa sigue disponible.

- **Saturación de actuadores.** El mezclador desatura con la prioridad de PX4: conserva primero el momento de roll y pitch, luego el empuje y por último la guiñada. Si aun así debe recortar roll y pitch, la interfaz lo avisa con el factor `k_rp` en ámbar.
- **PID con protecciones.** Derivada sobre la medida con filtro pasa-bajos, anti-windup por integración condicional y límite absoluto del integrador, como en PX4 y Betaflight.
- **Barridos tolerantes a cortes.** Cada corrida terminada se escribe en `runs.jsonl` con `fsync`; si el proceso se interrumpe, relanzarlo con la misma carpeta salta lo ya hecho y descarta una última línea truncada.
- **Detección de choques.** Los escenarios de barrido marcan como choque una inclinación mayor de 60°, una altura menor de 0.2 m o un valor no numérico; esas configuraciones reciben costo infinito y no contaminan el ranking.

6.2 Desempeño

La física corre a 1 kHz con RK4 y aun así el simulador completo (sensores, estimador, filtros, cascada de control, mezclador y perturbaciones) avanza a ≈13.5 veces el tiempo real sin interfaz, medido en el bloque L de la verificación numérica. En la interfaz, un temporizador de 50 Hz avanza la simulación con un presupuesto de 28 ms por cuadro: si la computadora no alcanza, el factor de tiempo real mostrado baja, pero la ventana no se congela. Las comparaciones y los barridos corren en hilos aparte, y los barridos pueden usar varios procesos.

Tabla 5 | Costo de cómputo por llamada del lazo externo (periodo de 10 ms), figura en 8 nominal, media de 3 semillas.

Controlador	Media [ms]	Percentil 95 [ms]	Máximo [ms]
PID en cascada	< 0.05	—	—
MPC lineal	0.37	0.54	2.2
MPC no lineal	4.01	5.70	16.1
RL puro	0.11	0.20	1.4
MPC + RL	0.50	0.74	4.3

6.3 Consistencia y reproducibilidad

Toda la aleatoriedad del simulador (ruido de sensores, sesgo del giroscopio, turbulencia, pulsos de interferencia) sale de generadores con semilla. Con la misma semilla dos corridas dan métricas idénticas bit a bit, dentro del mismo proceso y entre ejecuciones con uno o dos procesos; las pruebas automáticas lo comprueban. Los barridos usan números aleatorios comunes: todos los puntos de un barrido ven el mismo ruido para cada réplica, de modo que las diferencias se deben a los parámetros. La interfaz, el banco de comparación, los barridos y las pruebas inicializan el vuelo con la misma función (`scenarios.prepare_tracking_sim`).

7 Interfaz y usabilidad

La interfaz está organizada en seis áreas de trabajo, accesibles desde un riel lateral o con `Ctrl+1` a `Ctrl+6`: Actitud, Trayectorias, Perturbaciones, Sensores, Barridos y Modelo. Una barra superior concentra el vehículo, los controles de reproducción, la velocidad de simulación, el reloj simulado con el factor de tiempo real medido y el estimador activo; un registro de eventos plegable informa avisos y errores (figura 2). La barra de menús agrupa Archivo (abrir y guardar escenarios, exportar CSV), Simulación, Ver, Ejemplos y Ayuda (tour guiado, atajos, «Acerca de»). La primera vez que se abre, una ventana de bienvenida ofrece el tour o los ejemplos.

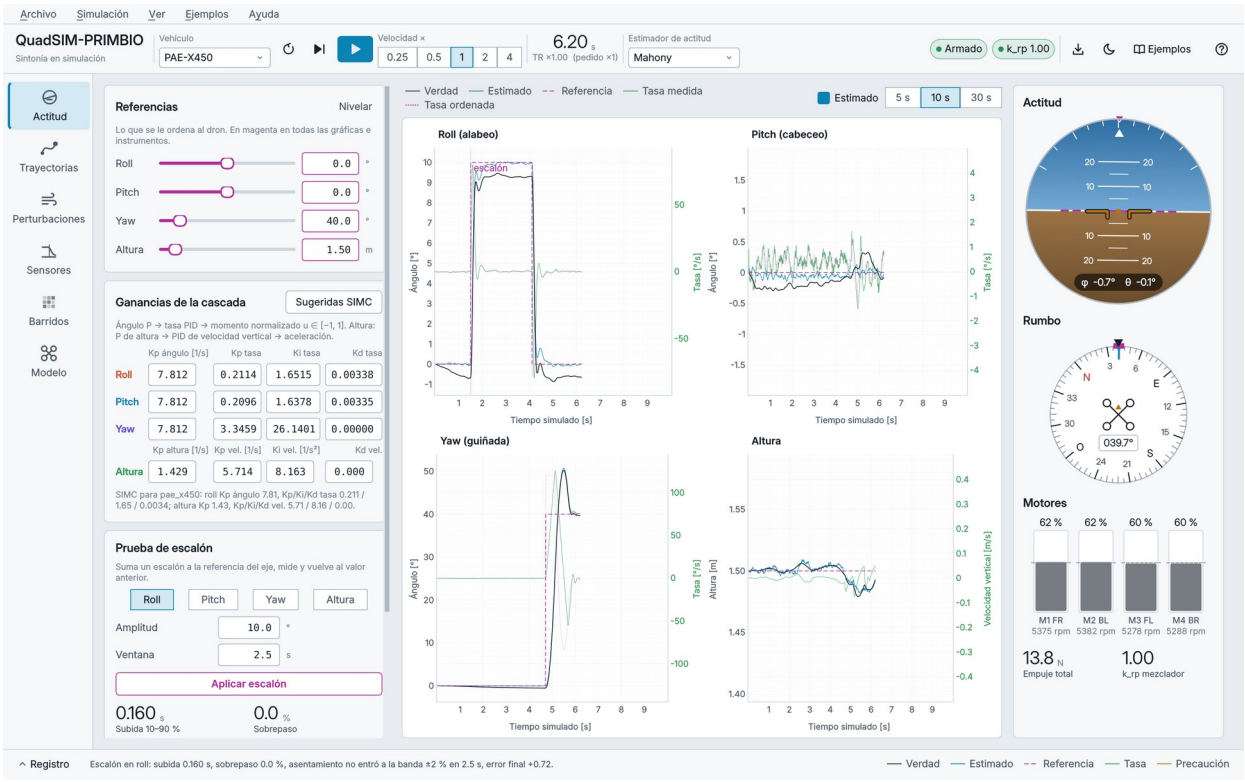


Figura 2 | Ventana principal en tema claro, página de actitud tras un escalón de 10° en roll. Captura de una corrida real.

El color tiene un significado fijo en todo el programa, tomado de las pantallas de vuelo de cabina: blanco (negro en tema claro) es lo que el dron hace, cian lo que cree el estimador, magenta lo que se le ordena, verde las tasas angulares, ámbar las precauciones y rojo las alarmas. Quien aprende a leer una gráfica ya sabe leer todas. Hay tema oscuro y tema claro (`Ctrl+T`), este último pensado para proyector e impresión.

> **NOTA** La descripción detallada de cada página, con capturas anotadas y ejemplos paso a paso, está en el Manual de Usuario (PQS-MU-003).

8 Integridad y seguridad de la información

QuadSIM-PRIMBIO es una aplicación local: no abre puertos, no se conecta a Internet durante su uso y no maneja datos personales. Por eso varios mecanismos propios de sistemas en red no aplican y se declaran como tales.

Tabla 6 | Aspectos de integridad y seguridad.

Aspecto	Aplica	Tratamiento en el software
Autenticación y perfiles	No	No hay usuarios ni cuentas; quien abre el programa tiene acceso completo.
Cifrado	No	Los archivos de resultados (CSV, JSON, NPZ) son datos experimentales no sensibles y se guardan sin cifrar.
Seguridad física	No	No controla hardware: un error de sintonía solo afecta al dron simulado.
Consistencia del estado	Sí	Un único objeto es dueño de la simulación; las páginas se enteran de los cambios por señales y no guardan referencias a objetos que el reinicio reemplaza.
Aislamiento de corridas	Sí	Comparaciones y barridos trabajan sobre copias profundas de la configuración; no alteran la simulación en vivo.
Integridad de resultados	Sí	Cada barrido guarda su especificación y una huella; una carpeta con un experimento distinto se rechaza en lugar de mezclarse.

Aspecto	Aplica	Tratamiento en el software
Trazabilidad	Sí	Semillas fijas, resultados crudos por corrida y parámetros del vehículo con su fuente.
Modelos entrenados	Sí	Las políticas de RL se guardan en archivos NPZ con los parámetros del modelo sustituto con que se entrenaron.

9 Portabilidad y compatibilidad

9.1 Plataformas compatibles

Tabla 7 | Compatibilidad por plataforma.

Plataforma	Estado
Windows 10/11	Plataforma de uso prevista. La interfaz usa escalado de alta densidad sin redondeo y tipografías de Windows (Bahnschrift, Segoe UI). Se distribuye además como ejecutable de PyInstaller (una carpeta con QuadSIM-PRIMBIO.exe), compilado en una máquina Windows de GitHub Actions o localmente con build_exe.bat.
Linux	Plataforma de desarrollo y de verificación: las pruebas, las capturas y los resultados de este documento se obtuvieron en Linux.
macOS	No verificado. Todas las dependencias tienen distribución para macOS, pero no se ha probado.

9.2 Dependencias relevantes

NumPy es la única dependencia del núcleo de simulación. La interfaz agrega PySide6, pyqtgraph y, de forma opcional, PyOpenGL. SciPy se usa en la identificación del modelo sustituto de aprendizaje por refuerzo y Matplotlib en las gráficas de los barridos fuera de la interfaz. No se usan bibliotecas de aprendizaje profundo: la red de las políticas y el algoritmo PPO están escritos en NumPy, con retropropagación manual verificada contra diferencias finitas.

9.3 Portabilidad

El paquete `quadsim/` es Python puro sobre NumPy y corre en cualquier sistema con esa biblioteca, con o sin pantalla. La interfaz depende de Qt 6, disponible en los tres sistemas de escritorio. Las rutas de archivos se construyen con `os.path` y los procesos paralelos usan el método de arranque `spawn`, que es el de Windows y es seguro dentro de una aplicación Qt con hilos.

10 Documentación y soporte técnico

Tabla 8 | Documentación disponible.

Recurso	Disponible	Contenido
Manual de Usuario (PQS-MU-003)	Sí	Instalación, recorrido por la interfaz, uso de cada página, ejemplos y solución de problemas.
Manual Técnico (PQS-MT-002)	Sí	Arquitectura, componentes, modelo, algoritmos, temporización y despliegue.
Modelo matemático, 3.ª ed.	Sí	55 páginas: ecuaciones, parámetros con fuente y verificación numérica (PDF y LaTeX).
Guías del repositorio	Sí	README, guía de la interfaz, ruta de presentación, barridos y aprendizaje por refuerzo (Markdown).
Código fuente comentado	Sí	Comentarios y docstrings en español con fuentes bibliográficas.
Pruebas automáticas	Sí	223 pruebas con pytest.
Ejecutable para Windows	Sí	Carpeta autocontenida generada con PyInstaller (QuadSIM-PRIMBIO.exe).

Recurso	Disponible	Contenido
Ayuda en la aplicación	Sí	Tour guiado, galería de ejemplos con resultado esperado, atajos y «Acerca de».
Repositorio	Sí	github.com/JesusJacome18/PAE-Quadcopter-SIM

10.1 Soporte técnico

El Manual de Usuario incluye una sección de solución de problemas organizada por síntoma. Las consultas se atienden en el repositorio del proyecto y en los correos de los autores (Jesús Daniel Jácome: jejacome@unal.edu.co · Francisco Javier Burgos Flórez: fjburgosf@unal.edu.co).

11 Pruebas, validación y desempeño

La validación se apoya en tres niveles: pruebas automáticas del código, verificación numérica del modelo contra la teoría y bancos de comparación de controladores con varias semillas. Ningún resultado de este documento se escribió a mano; todos salen de esos guiones.

11.1 Pruebas automáticas

Tabla 9 | Pruebas automáticas por archivo (pytest).

Archivo	Pruebas	Qué verifica
tests/test_audit_actuation.py	9	Retardo de cómputo, ESC con muestreo y retención, cuantización del acelerador y ganancias con la cadena configurada.
tests/test_audit_core.py	17	Base de tiempo con razones no enteras, líneas de retardo, registro, semillas y referencia por instante.
tests/test_audit_estimation.py	16	Compensación de aceleración (vuelo solo con barómetro, rampa, recorte), validación de estimadores y magnetómetro solo para el rumbo.
tests/test_audit_rl_nmpc.py	10	Regularización del NMPC, carga y validación de políticas, sustituto versionado y estadística del banco.
tests/test_gui_examples.py	16	Cada ejemplo carga, abre su página y configura lo que dice su archivo; cifras de los ejemplos 01, 02 y 08.
tests/test_gui_scenarios.py	14	Guardar y abrir escenarios, archivos inválidos, menús y atajos, diálogos y cada botón visible.
tests/test_gui_smoke.py	15	Interfaz sin pantalla: páginas, gráficas, filtros, comparación y exportación.
tests/test_gui_tour.py	6	Tour completo paso a paso, teclado, cambio de tamaño y de tema, tours por página y bienvenida.
tests/test_position_controllers.py	24	Registro, esquema de parámetros y seguimiento de cada controlador de posición.
tests/test_qa_funciones.py	8	Lazo de altura en la página Actitud y modos de cámara de la escena 3D.
tests/test_qa_robustez.py	34	Regresiones de la interfaz con entradas extremas y cambios a mitad de corrida.
tests/test_rl.py	12	Gradientes de la red contra diferencias finitas, entorno vectorizado, políticas y controladores.
tests/test_sweeps.py	19	Rutas de parámetros, diseños, determinismo, procesos paralelos, reanudación y frente de Pareto.
tests/test_trajectories.py	23	Derivadas analíticas contra diferencias finitas, convención NED, extremos de las trayectorias.
Total	223	

La última ejecución completa registrada (2026-10-10 10:15, Python 3.13.16) terminó con 219 pruebas aprobadas y 0 fallidas en 230 s. Otras 4 pruebas, que miden tiempo de cómputo y dependen del equipo, se omiten por defecto y se activan con la variable de entorno PERF_TESTS=1.

11.2 Verificación numérica del modelo

El guion `experiments/model_verification.py` (bloques A a M) compara cada parte del modelo con un resultado conocido. La tabla 10 recoge algunas cifras; el documento del modelo presenta todas.

Tabla 10 | Extracto de la verificación numérica.

Prueba	Resultado	Referencia
Orden de convergencia de RK4	4.19	4 (teórico)
Orden de convergencia de Euler	1.00	1 (teórico)
Deriva relativa de energía, rotación libre (RK4)	2.7×10^{-14}	Euler: 6.0×10^{-3}
Inestabilidad del eje intermedio (Dzhanibekov), primera inversión	7.87 s	8.53 s (estimación)
Acelerador de vuelo estacionario, PAE-X450	0.609	0.609 (fuente, Init.m)
Pico del espectro del giroscopio en estacionario	177.2 Hz	177.4 Hz (paso de pala calculado)
Error de roll en lazo cerrado, RK4 a 1 kHz	1.4×10^{-7}°	Referencia RK4 a 16 kHz
Escalón de 10° con ganancias SIMC: subida 10–90 %	0.13 s	0.128 s (diseño)
Dryden: correlación medida vs. teórica	3.75 s	3.89 s
Factor de tiempo real (estimador Mahony)	13.5×	—

11.3 Banco de comparación de controladores

El guion `experiments/benchmark_controllers.py` corrió los cinco controladores sobre cuatro trayectorias y cuatro escenarios de perturbación, con tres semillas cada uno, todos con el vehículo PAE-X450 y el estimador Mahony. La tabla 11 muestra el error medio cuadrático de seguimiento en 3D (media ± desviación estándar muestral entre semillas). La negrita marca la media más baja de cada fila y nada más: con tres semillas los rangos de MPC, MPC no lineal y MPC + RL se solapan en las filas de figura en 8 y círculo.

Tabla 11 | RMSE 3D de seguimiento [cm] por controlador.

Trayectoria · escenario	PID	MPC	MPC no lineal	RL puro	MPC + RL
Figura en 8 (periodo 10 s) · nominal	1.83 ± 0.28	**1.50 ± 0.36**	1.58 ± 0.41	4.56 ± 0.19	1.79 ± 0.37
Figura en 8 (periodo 10 s) · viento	1.84 ± 0.44	**1.66 ± 0.42**	1.71 ± 0.46	5.36 ± 0.45	1.88 ± 0.44
Figura en 8 (periodo 10 s) · carga de 300 g	2.23 ± 0.28	**1.85 ± 0.32**	2.05 ± 0.37	4.66 ± 0.42	2.06 ± 0.29
Figura en 8 (periodo 10 s) · interferencia magnética	5.90 ± 2.82	**3.00 ± 0.14**	3.03 ± 0.18	6.27 ± 0.09	3.13 ± 0.16
Figura en 8 rápida (periodo 5 s) · nominal	6.22 ± 0.10	**2.53 ± 0.24**	2.87 ± 0.27	9.36 ± 0.23	3.07 ± 0.20
Figura en 8 rápida (periodo 5 s) · viento	5.54 ± 0.12	**2.32 ± 0.27**	2.62 ± 0.29	10.22 ± 0.48	2.74 ± 0.21
Figura en 8 rápida (periodo 5 s) · carga de 300 g	7.28 ± 0.16	**4.18 ± 0.29**	4.22 ± 0.31	9.28 ± 0.25	4.46 ± 0.26
Figura en 8 rápida (periodo 5 s) · interferencia magnética	19.15 ± 2.90	16.27 ± 0.49	**15.23 ± 0.41**	138.91 ± 8.76	16.78 ± 0.50
Círculo · nominal	1.59 ± 0.36	**1.44 ± 0.39**	1.52 ± 0.45	3.93 ± 0.26	1.77 ± 0.40
Círculo · viento	1.80 ± 0.40	**1.61 ± 0.36**	1.65 ± 0.43	4.60 ± 0.70	1.85 ± 0.38
Círculo · carga de 300 g	1.98 ± 0.34	**1.72 ± 0.36**	1.96 ± 0.40	4.24 ± 0.49	1.99 ± 0.32
Círculo · interferencia magnética	4.19 ± 1.80	**1.81 ± 0.28**	1.91 ± 0.31	3.48 ± 0.23	1.91 ± 0.33
Escalones (caja) · nominal	28.13 ± 0.20	**8.87 ± 0.06**	9.02 ± 0.07	20.88 ± 0.09	8.99 ± 0.05
Escalones (caja) · viento	27.85 ± 0.23	**8.94 ± 0.08**	9.09 ± 0.10	21.07 ± 0.12	8.99 ± 0.07
Escalones (caja) · carga de 300 g	28.86 ± 0.25	**9.32 ± 0.06**	9.55 ± 0.08	21.09 ± 0.10	9.66 ± 0.05

Trayectoria · escenario	PID	MPC	MPC no lineal	RL puro	MPC + RL
Escalones (caja) · interferencia magnética	133.44 ± 2.87	**13.68 ± 0.44**	14.90 ± 0.53	21.21 ± 0.12	14.50 ± 0.60

Fuente: experiments/results/benchmark_final/summary.md (generado por el guion; 3 semillas por celda).

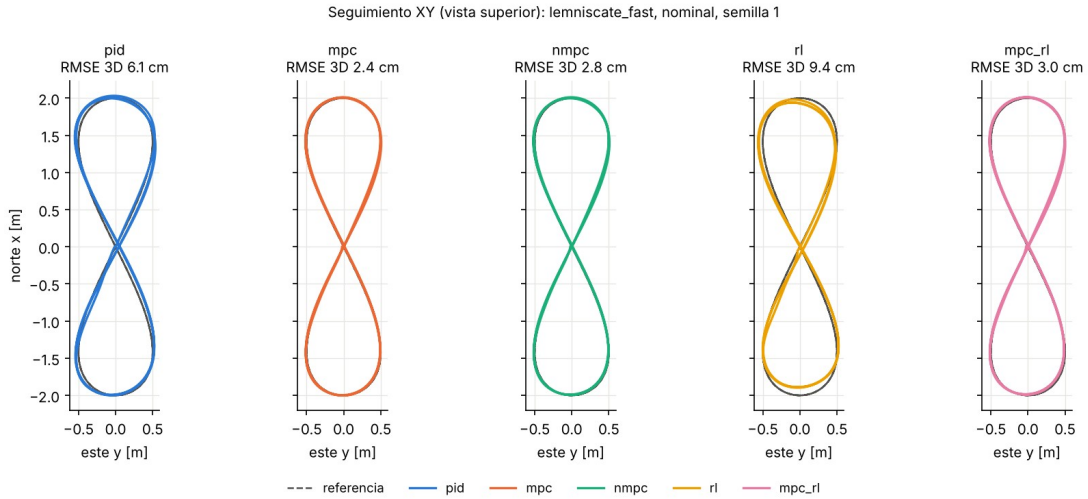


Figura 3 | Vista superior de la figura en 8 rápida (periodo 5 s), escenario nominal, semilla 1: referencia en gris y trayectoria de cada controlador.

La comparación más informativa es por semilla, porque cada semilla da a todos los controladores el mismo ruido y la misma turbulencia. Con ese criterio el MPC lineal queda por debajo del PID en 48 de las 48 parejas (fila por semilla), y el MPC + RL queda por encima del MPC en 48 de 48. Aun cuando las tres semillas coinciden, una prueba de signos da p = 0.25 (bilateral): son tendencias consistentes, no diferencias estadísticamente probadas. En la figura en 8 rápida el MPC reduce el error del PID en 59 %, porque anticipa la referencia en lugar de reaccionar a ella.

La fila más exigente es la figura en 8 rápida con interferencia magnética: allí el RL puro pierde la trayectoria (error máximo de más de 1 m) y el de menor error medio es el MPC no lineal, con 15.2 cm. El resultado de los demás depende de una decisión de diseño del filtro de navegación: su compuerta de innovación rechaza una medida de posición solo si es improbable (más de 5 desviaciones estándar) y además mayor que 25 cm. Con el rumbo desviado por la interferencia, la predicción queda sesgada y una compuerta puramente estadística descartaría medidas correctas una tras otra; con ella, el MPC llega a 1.6 m de RMSE en esa fila.

El aprendizaje por refuerzo puro, entrenado fuera de línea, no alcanzó al MPC: en las 15 filas en que no pierde la trayectoria, su RMSE medio va de 1.6 a 4.4 veces el del MPC y por semilla llega a 4.6 veces. Superó al PID en 5 de las 16 filas. La mayor parte de su error ya aparece en el modelo sustituto donde se entrenó (4.25 cm frente a 0.14 cm del MPC en la figura en 8), de modo que la brecha se debe sobre todo a la política, su entrenamiento y su diseño; qué dinámicas faltan en el sustituto sigue siendo una hipótesis. El residuo aprendido sobre el MPC lo empeora incluso en el sustituto, porque el punto guardado se eligió por un costo que premia la suavidad además del error. Los controladores clásicos se afinaron con una de las semillas del banco, ventaja que el RL no tuvo. El Manual Técnico (PQS-MT-002) y el documento del modelo detallan estas condiciones.

12 Impacto y utilización

El software se desarrolló dentro del Semillero PRIMBIO para que sus resultados se usen en tres frentes de la universidad.

12.1 Docencia

En un curso de control, el simulador permite que un estudiante vea en segundos lo que en un banco real cuesta una tarde: qué le pasa al sobrepaso cuando se triplica la ganancia del lazo de ángulo, por qué un lazo interno débil arruina la cascada o cuánto retardo agrega un filtro de 20 Hz. El repositorio incluye una ruta de presentación de unos 40 minutos con casos de prueba y resultados esperados, que sirve de guion para una práctica de laboratorio.

12.2 Investigación

El banco de comparación produce tablas reproducibles con varias semillas, escenarios de perturbación estándar y métricas de seguimiento, esfuerzo y cómputo. Es la base para comparar contra la literatura técnicas de control predictivo, control no lineal y aprendizaje por refuerzo, y para estudiar la brecha entre un modelo de entrenamiento simplificado y la planta completa.

12.3 Extensión

Un simulador en español, construido sobre bibliotecas de código abierto y sin requisitos de hardware especial, puede llevarse a colegios, ferias y semilleros de otras sedes para mostrar cómo vuela y cómo se controla un dron, con un vehículo virtual que no se rompe.

12.4 Usuarios y utilización

Los usuarios directos son los integrantes del semillero y los estudiantes de Ingeniería Mecatrónica de la Sede La Paz. El flujo de trabajo para el que se diseñó, del modelo al dron real, se muestra en la figura 4.

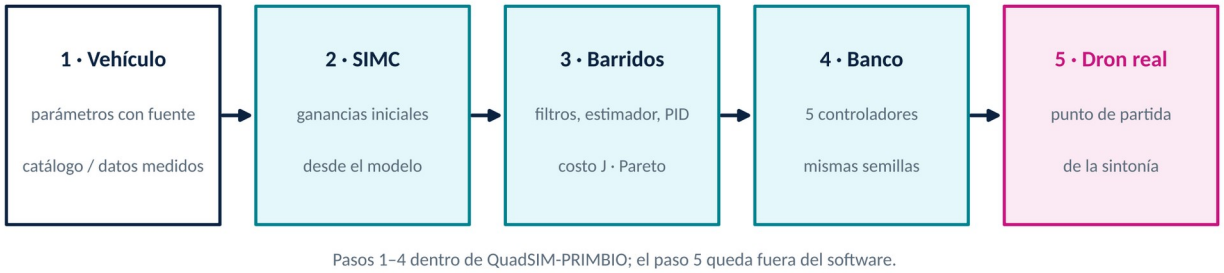


Figura 4 | Flujo de sintonía propuesto: los pasos 1 a 4 se hacen en el simulador.

13 Contribución al estado del arte

13.1 Problema abordado

Sintonizar el control de un multirrotor exige ajustar decenas de parámetros acoplados: ganancias de dos lazos por eje, frecuencias de corte de filtros, ganancias de estimadores y, si se usa un controlador de trayectoria, sus pesos y horizontes. Hacerlo directamente en el vehículo es lento y riesgoso. La alternativa obvia, sintonizar en simulación, solo funciona si el simulador contiene los fenómenos que limitan al controlador real: el rezago de los motores, la saturación de los actuadores, el acoplamiento entre inclinación y traslación, la vibración que ven los sensores, el error del estimador y el viento.

13.2 Estado del arte

Los textos de Quan [1, 2] y el simulador CopterSim [3], base de la plataforma RflySim, ofrecen un modelo completo de multirrotor con parámetros de un vehículo real, orientado a MATLAB/Simulink y a la prueba de firmware PX4. Powers, Mellinger y Kumar [4] resumen la cinemática y la dinámica del cuadricóptero que usan

la mayoría de los trabajos de control. Entre los simuladores de código abierto, RotorS [5] se apoya en Gazebo y ROS para simular vehículos aéreos con sus sensores; AirSim [6] y Flightmare [7] priorizan la percepción visual con motores gráficos de alto realismo; y gym-pybullet-drones [8] ofrece entornos de PyBullet para aprendizaje por refuerzo multiagente.

En la estimación de actitud, los filtros de Madgwick [9] y Mahony [10] y el filtro de Kalman extendido con cuaterniones [11] son las referencias habituales; Leishman y colaboradores [12] mostraron que el acelerómetro de un multirrotor no mide la gravedad cuando el vehículo acelera, un efecto que muchos simuladores omiten. En control de trayectoria, la planitud diferencial y la generación de trayectorias de mínimo snap [13] son la base del seguimiento agresivo; Sun y colaboradores [14] compararon MPC no lineal y control basado en planitud en vuelo ágil, y PPO [15] es el algoritmo de aprendizaje por refuerzo más usado para control de multirrotores. Para la sintonía de PID a partir de un modelo, las reglas SIMC de Skogestad [16] dan ganancias iniciales con un solo parámetro de diseño.

En la Universidad Nacional de Colombia, Quitián Niño [17] construyó bancos de prueba para medir los parámetros del modelo dinámico de drones, Larotta Mariño [18] desarrolló el control de posición de un cuadricóptero con realidad virtual y Casallas Moreno [19] propuso un modelo digital de un multirrotor con control distribuido.

13.3 Limitaciones actuales del estado del arte

- Los simuladores de alto realismo visual necesitan motores gráficos, ROS o tarjetas de video dedicadas, y su ciclo de simulación no está pensado para correr miles de veces en un barrido de parámetros.
- Los entornos para aprendizaje por refuerzo suelen simplificar sensores, estimación y control de bajo nivel, de modo que la política aprende sobre una planta más dócil que la real.
- Las comparaciones entre controladores publicadas usan condiciones distintas (vehículo, ruido, perturbaciones, inicialización), lo que dificulta saber si una diferencia se debe al controlador.
- Pocas herramientas muestran juntos, en vivo, el espectro de los sensores, la respuesta de los filtros, el error del estimador y las métricas de control, y casi ninguna está en español para la docencia local.

13.4 Aporte de QuadSIM-PRIMBIO

El aporte no está en un algoritmo nuevo sino en la integración verificada. El software reúne en una sola herramienta en Python, sin motor gráfico ni ROS, un modelo 6-DOF con parámetros trazables, sensores con la vibración real de las hélices, tres estimadores, el control en cascada de un autopiloto, cinco controladores de posición comparados con la misma semilla, perturbaciones estándar y un motor de barridos con criterios estadísticos (números aleatorios comunes, réplicas, frente de Pareto). Cada cifra del documento del modelo se recalcula con un guion, y el resultado negativo del aprendizaje por refuerzo quedó medido y reportado con sus límites en lugar de omitirse. Para el semillero, el simulador deja listo el camino hacia el dron real: medir sus parámetros con el protocolo de bancos de prueba ya publicado en la universidad, cargarlos como un vehículo nuevo y repetir sobre él el flujo de sintonía que este documento describe.

14 Aportes de los autores

Jesús Daniel Jácome. Concepción del simulador, modelado matemático, programación del núcleo de simulación, de los controladores y de la interfaz gráfica, verificación numérica, ejecución de los bancos de comparación y redacción de la documentación técnica.

Francisco Javier Burgos Flórez. Dirección del proyecto: definición de la orientación de investigación (sintonía en entornos simulados como punto de partida para drones reales), de las técnicas de control y de las perturbaciones que debía evaluar el simulador, y revisión de los resultados.

15 Referencias

[1] Quan, Q. (2017). *Introduction to Multicopter Design and Control*. Springer, Singapur.

[2] Quan, Q., Dai, X. y Wang, S. (2020). *Multicopter Design and Control Practice: A Series Experiments Based on MATLAB and Pixhawk*. Springer, Singapur.

[3] Dai, X. CopterSim, archivo `Init.m`. Repositorio github.com/XunhuaDai/CopterSim, commit 0d14487f (2019).

[4] Powers, C., Mellinger, D. y Kumar, V. (2015). Quadrotor kinematics and dynamics. En *Handbook of Unmanned Aerial Vehicles*, cap. 16, pp. 307–328. Springer.

[5] Furrer, F., Burri, M., Achtelik, M. y Siegwart, R. (2016). RotorS — A modular Gazebo MAV simulator framework. En *Robot Operating System (ROS): The Complete Reference*, vol. 1, pp. 595–625. Springer.

[6] Shah, S., Dey, D., Lovett, C. y Kapoor, A. (2018). AirSim: High-fidelity visual and physical simulation for autonomous vehicles. En *Field and Service Robotics*, pp. 621–635. Springer.

[7] Song, Y., Naji, S., Kaufmann, E., Loquercio, A. y Scaramuzza, D. (2020). Flightmare: A flexible quadrotor simulator. *Conference on Robot Learning (CoRL)*.

[8] Panerati, J., Zheng, H., Zhou, S., Xu, J., Prorok, A. y Schoellig, A. P. (2021). Learning to fly: a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control. *IEEE/RSJ IROS*.

[9] Madgwick, S. O. H., Harrison, A. J. L. y Vaidyanathan, R. (2011). Estimation of IMU and MARG orientation using a gradient descent algorithm. *IEEE ICORR*.

[10] Mahony, R., Hamel, T. y Pflimlin, J.-M. (2008). Nonlinear complementary filters on the special orthogonal group. *IEEE Transactions on Automatic Control*, 53(5), 1203–1218.

[11] Sabatini, A. M. (2006). Quaternion-based extended Kalman filter for determining orientation by inertial and magnetic sensing. *IEEE Transactions on Biomedical Engineering*, 53(7), 1346–1356.

[12] Leishman, R. C., Macdonald, J. C., Beard, R. W. y McLain, T. W. (2014). Quadrotors and accelerometers: State estimation with an improved dynamic model. *IEEE Control Systems Magazine*, 34(1), 28–41.

[13] Mellinger, D. y Kumar, V. (2011). Minimum snap trajectory generation and control for quadrotors. *IEEE ICRA*, 2520–2525.

[14] Sun, S., Romero, A., Foehn, P., Kaufmann, E. y Scaramuzza, D. (2022). A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. *IEEE Transactions on Robotics*, 38(6), 3357–3373.

[15] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. y Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

[16] Skogestad, S. (2003). Simple analytic rules for model reduction and PID controller tuning. *Journal of Process Control*, 13(4), 291–309.

[17] Quitián Niño, L. C. (2022). *Construcción de bancos de prueba para la medición experimental de los parámetros del modelo dinámico de drones – UAS*. Tesis de maestría, Universidad Nacional de Colombia, Bogotá.

[18] Larotta Mariño, L. A. (2024). *Control de posición para un cuadricóptero basado en un sistema de realidad virtual*. Tesis de maestría, Universidad Nacional de Colombia, Bogotá.

[19] Casallas Moreno, E. A. (2025). *Modelo digital VMD de un sistema multirrotor con enfoque de control distribuido*. Tesis de maestría, Universidad Nacional de Colombia, Bogotá.

[20] Jesús Daniel Jácome (2026). *Modelo matemático del simulador (repositorio PAE Quadcopter SIM), 3.ª edición*. Universidad Nacional de Colombia, Sede La Paz. docs/Modelo_Matematico_Simulador_3ra_Edicion.pdf.